MALICK BUSS SÉNÉGAL

Plateforme E-Commerce Marketplace Multi-Boutiques

Architecture Mobile-First • Souveraineté FCFA • Écosystème Déployé Hybride

---

Dossier Global d'Ingénierie Logicielle, Référentiel des Use Cases, Sécurité & Guide d'Exploitation en Production

---

Auteurs & Conception :
Direction Technique & Ingénierie
Architecture Cloud & DevOps
Malick Buss Engineering Team

Statut : Production Active
Classification : Confidentiel Entreprise
Version : 2.0.0 (Consolidée)
Publication : Novembre 2026

Environnements Déployés & Opérationnels :
Vitrine Publique (Vercel) : https://frontend-malickdevwebs-projects.vercel.app
Super Admin (Vercel) : https://back-office-malickdevwebs-projects.vercel.app
API Backend (Render) : https://malick-buss-api.onrender.com
Base de Données : Neon Serverless PostgreSQL (TLS / Pooled)

Siège Opérationnel : Dakar, République du Sénégal — Contact : contact@malickbuss.sn

Fiche Signalétique du Projet

Paramètre Clé	Spécification Technique et Métier
Désignation Officielle	Marketplace Multi-Boutiques & Passerelle E-Commerce Malick Buss
Couverture Géographique	Sénégal (Dakar & les 13 régions de l'intérieur), extension UEMOA
Monnaie de Référence	Franc CFA (XOF / FCFA), précision entière & centimes comptables stricts
Paiements Mobiles	Wave Senegal API, Orange Money Webhook / OTP, Cash On Delivery (COD)
Communication Client	Passerelle directe WhatsApp (+221) via deep-linking `wa.me` sécurisé
Architecture Monorepo	Monorepo TypeScript modulaire (`shared`, `backend`, `frontend`, `back-office`)
Moteur Backend	NestJS 10, Clean Architecture, Domain-Driven Design (DDD), Prisma ORM 5.22
Persistance & Données	Neon Serverless PostgreSQL 16 (SSL obligatoire, PgBouncer) + Cache Redis 7
Frontends Applicatifs	Next.js 14 App Router, React 18, Tailwind CSS, Framer Motion (Mobile-First)
Authentification Médias	Cloudinary Media API avec contrainte de capture caméra directe (`liveCapture`)
Hébergement Hybride	Frontends sur Vercel Edge CDN, Backend API sur Render Cloud (Frankfurt)
Dépôt de Code Source	GitHub : `https://github.com/MalickDevWeb/malick-221` (Branche `main`)
Contrôle Qualité & CI/CD	SonarQube Quality Gate bloquant, LCOV Coverage $> 80\%$, GitHub Actions

TABLE 1 – Fiche Signalétique et Spécifications Générales du Projet Malick Buss

Table des matières

Fiche Signalétique du Projet **1**

1 Vision Stratégique, Contexte de Marché & Modèle Économique **5**
1.1 Contexte Économique du Commerce au Sénégal 5
1.2 Les Quatre Piliers Fondateurs de la Solution Malick Buss 5
1.3 Modèle de Rôles & Contrôle d'Accès (RBAC) 6

2 Architecture Monorepo, Stack Technique & Sécurité **7**
2.1 Topologie du Monorepo TypeScript 7
2.2 Sécurisation HTTP, En-têtes & Résolution Dynamique CORS 7
2.3 Sondes de Santé & Résilience des Conteneurs 7

3 Architecture des Données Relationnelles & Moteur Métier **9**
3.1 Modélisation Prisma ORM & PostgreSQL 9
3.2 Moteur de Calcul des Soldes & Grand Livre des Commissions 9

4 Référentiel Exhaustif des 40 Use Cases du Super Admin **11**
4.1 Domaine 1 : Pilotage Stratégique & Tableau de Bord 11
4.2 Domaine 2 : Onboarding & Validation KYC des Boutiques 11
4.3 Domaines 3 à 8 : Gestion Opérationnelle, Catalogue & Finance 12

5 Qualité Logicielle, Pipeline CI/CD & SonarQube **13**
5.1 Architecture Globale CI/CD & Bouclier Qualité 13
5.2 Exigences du Quality Gate SonarQube 13

6 Topologie de Déploiement Hybride & Écosystème Sénégal **14**
6.1 Schéma Structurel du Flux Opérationnel 14
6.2 Hébergement Hybride Déployé en Production 14

7 Galerie Visuelle des Interfaces & Maquettes **16**
7.1 Parcours Acheteur & Espace Marchand Mobile-First 16
7.2 Tableau de Bord Vendeur & Modération Super Admin 17

8 Manuel d'Exploitation, Commandes & Plan de Continuité **19**
8.1 Répertoire des Commandes Opérationnelles 19
8.2 Plan de Continuité d'Activité (PCA) & Reprise d'Urgence 19

Table des figures

2.1 Code Source : Configuration des En-têtes Helmet & Résolution Dynamique des Origines CORS 8
2.2 Code Source : Implémentation des Sondes de Disponibilité Liveness & Readiness . . 8

3.1 Code Source : Service Métier de Publication de Produits & Contrôle des Stocks . . . 10
3.2 Code Source : Agrégation Transactionnelle du Solde Marchand & Gestion des Retraits 10

5.1 Architecture Globale Cloud, Déploiement CI/CD & Bouclier Quality Gate SonarQube 13

6.1 Schéma Structurel du Flux Opérationnel en 5 Phases d'Exécution 14

7.1 Maquette Interface Utilisateur : Parcours d'Achat & Découverte Produits 16
7.2 Maquette Interface Marchand : Gestion des Ventes, Stocks & Demandes de Retrait . 17
7.3 Console Super Admin Mobile : Validation KYC & Arbitrage des Retraits de Fonds . 18

Liste des tableaux

1 Fiche Signalétique et Spécifications Générales du Projet Malick Buss 1

1.1 Matrice des Rôles et Droits Applicatifs (RBAC) 6

4.1 Référentiel Use Cases — Domaine 1 : Pilotage Stratégique 11
4.2 Référentiel Use Cases — Domaine 2 : Onboarding & Vérification Marchande 11
4.3 Référentiel Synthétique — Domaines 3 à 8 12

8.1 Manuel des Commandes d'Exploitation Quotidienne 19

Chapitre 1

Vision Stratégique, Contexte de Marché & Modèle Économique

1.1 Contexte Économique du Commerce au Sénégal

Le commerce de détail au Sénégal repose à plus de 70% sur le secteur informel et de proximité. Bien que l'accès aux terminaux mobiles 4G/5G et aux services financiers numériques tels que Wave et Orange Money soit aujourd'hui quasi universel dans les centres urbains (Dakar, Thiès, Saint-Louis, Touba), le commerce électronique informel (réseaux sociaux, messageries privées) fait face à des frictions critiques :

1. **L'asymétrie d'information et la méfiance visuelle :** Prolifération de visuels non représentatifs ou récupérés sur Internet. L'acheteur sénégalais exige de voir le produit réel avant de s'engager.
2. **L'opacité logistique et la dispersion tarifaire :** Absence de calcul forfaitaire unifié pour l'acheminement inter-urbain, générant des annulations de commande au moment de la livraison.
3. **L'absence d'outils de gestion pour les marchands :** Gestion manuelle de trésorerie sur cahiers papier, retards de rapprochement bancaire et litiges non documentés.

1.2 Les Quatre Piliers Fondateurs de la Solution Malick Buss

1. **Confiance Visuelle & Caméra Directe (*Live Capture*) :** La plateforme intègre un verrouillage de publication d'image. Lorsque l'option de certification est activée pour une boutique, le gérant ne peut pas téléverser de photos issues de sa galerie : l'application déclenche obligatoirement le capteur photo du smartphone en direct, garantissant l'authenticité physique des articles en stock.
2. **Souveraineté Monétaire FCFA & Prélèvement Automatisé :** Tous les prix, paniers et transactions sont libellés en FCFA. À chaque vente confirmée, l'algorithme prélève automatiquement la commission marketplace de 10% et crédite le solde disponible du marchand, débloquant des demandes de virement dès 5 000 FCFA.
3. **Ventilation Multi-Boutiques des Paniers (*Order Splitting*) :** L'acheteur peut constituer un panier unique comprenant des articles issus de marchands distincts. Le système scinde la commande en sous-commandes indépendantes (*SubOrders*), appliquant les frais logistiques fixes (3 000 FCFA) par boutique tout en assurant un suivi de livraison étanche.
4. **Intégration Fluide WhatsApp & Support In-App :** Un widget flottant persistant et des déclencheurs wa.me permettent à l'acheteur d'interagir directement avec le support ou le marchand via un lien contextualisé incluant le numéro de commande et le montant en FCFA.

1.3 Modèle de Rôles & Contrôle d'Accès (RBAC)

Rôle	Périmètre d'Accès	Privilèges Opérationnels
CLIENT	Vitrine & Espace Perso	Navigation, recherche, géolocalisation des boutiques, paiement Wave/OM/COD, suivi en direct, réclamations sous 7 jours.
VENDOR	Espace Boutiquier	Gestion des produits, stocks, mise à jour des statuts de livraison, consultation du grand livre comptable et demandes de retrait.
SUPER_ADMIN	Console d'Administration	Validation KYC des dossiers marchands, activation du flag caméra, modération du catalogue, arbitrage financier et paramétrage plateforme.

Table 1.1 – Matrice des Rôles et Droits Applicatifs (RBAC)

Chapitre 2

Architecture Monorepo, Stack Technique & Sécurité

2.1 Topologie du Monorepo TypeScript

Le code source est unifié dans un monorepo moderne géré par npm workspaces :

— **`shared/`** : Bibliothèque partagée TypeScript compilée en amont (`tsc`). Elle encapsule la fonction universelle `formatFCFA`, les validateurs de numéros sénégalais (+221) et les DTOs communs.

— **`backend/`** : API REST bâtie sur NestJS 10, suivant les principes de la *Clean Architecture* et du *Domain-Driven Design* (DDD).

— **`frontend/`** : Vitrine e-commerce cliente et portail d'administration pour les vendeurs, développés avec Next.js 14 App Router.

— **`back-office/`** : Console Super Admin dédiée à la modération, à la conformité financière et aux indicateurs de pilotage.

2.2 Sécurisation HTTP, En-têtes & Résolution Dynamique CORS

L'API implémente un module de filtrage strict autorisant dynamiquement les requêtes provenant des applications Vercel déployées et des environnements locaux autorisés :

2.3 Sondes de Santé & Résilience des Conteneurs

Pour garantir une haute disponibilité sans interruption de service lors des redéploiements, l'API NestJS expose deux sondes spécialisées :

— **`GET /health/live`** : Vérifie que le thread principal de Node.js répond correctement.

— **`GET /health/ready`** : Exécute une requête `SELECT 1` sur la base PostgreSQL pour s'assurer que la base de données est opérationnelle avant d'autoriser le trafic externe.

main.ts — Sécurité HTTP & CORS Dynamique

```ts
const app = await NestFactory.create(AppModule, { bodyParser: false });

const expressApp = app.getHttpAdapter().getInstance();
expressApp.disable('x-powered-by');

app.use((_req: any, res: any, next: () => void) => {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('X-Frame-Options', 'SAMEORIGIN');
  res.setHeader('X-XSS-Protection', '1; mode=block');
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
  next();
});

app.use(json({ limit: '5mb' }));
app.use(urlencoded({ extended: true, limit: '5mb' }));

const configuredOrigins = (
  process.env.CORS_ORIGINS
    ?? process.env.FRONTEND_CLIENT_URL
    ?? 'http://localhost:3000,http://localhost:3001,http://127.0.0.1:3000,http://127.0.0.1:3001'
).split(',').map((origin) => origin.trim()).filter(Boolean);

app.enableCors({
  origin: (requestOrigin, callback) => {
    if (!requestOrigin) return callback(null, true);
    const isAllowed = configuredOrigins.some((allowed) => {
      if (allowed === '*' || allowed === requestOrigin) return true;
      if (allowed.startsWith('*.')) {
        const domain = allowed.slice(2);
        return requestOrigin.endsWith(domain);
      }
      return false;
    }) || (process.env.NODE_ENV !== 'production' && requestOrigin.includes('localhost')) || requestOrigin.endsWith('.vercel.app');

    if (isAllowed) {
```

FIGURE 2.1 – Code Source : Configuration des En-têtes Helmet & Résolution Dynamique des Origines CORS

health.controller.ts — Sondes de Santé Liveness & Readiness

```ts
import { Controller, Get } from '@nestjs/common';
import { PrismaService } from '../../database/prisma.service';

@Controller('health')
export class HealthController {
  constructor(private readonly prisma: PrismaService) {}

  @Get('live')
  live() {
    return { status: 'ok' };
  }

  @Get('ready')
  async ready() {
    await this.prisma.$queryRaw`SELECT 1`;
    return { status: 'ok', database: 'ok' };
  }
}
```

FIGURE 2.2 – Code Source : Implémentation des Sondes de Disponibilité Liveness & Readiness

Chapitre 3

Architecture des Données Relationnelles & Moteur Métier

3.1 Modélisation Prisma ORM & PostgreSQL

Le schéma de base de données modélise les flux transactionnels complexes en garantissant la cohérence financière :

- `User` : Identité des utilisateurs avec hash de mot de passe dérive par `scrypt` et horodatage des sessions.
- `Shop` : Dossier marchand contenant les statuts KYC (`PENDING_VERIFICATION`, `ACTIVE`, `SUSPENDED`, `REJECTED`), le numéro de téléphone vérifié et la politique de capture photo.
- `Product` : Articles avec gestion des stocks physiques et réservations temporaires (15 minutes).
- `Order` & `SubOrder` : Structure en deux niveaux permettant à une commande globale d'être scindée par boutique, avec calcul dédié des frais de port et statut d'acheminement indépendant.
- `WithdrawalRequest` : Enregistrement immuable des demandes de retrait avec validation administrative obligatoire.

3.2 Moteur de Calcul des Soldes & Grand Livre des Commissions

Le calcul du solde marchand s'effectue par agrégation transactionnelle sécurisée. Seules les sous-commandes confirmées génèrent du solde retirable :

products.service.ts — Publication Sécurisée des Produits

```ts
    const slugBase = input.title
      .toLowerCase()
      .normalize('NFD')
      .replace(/[\u0300-\u036f]/g, '')
      .replace(/[^a-z0-9]+/g, '-')
      .replace(/(^-|-$)/g, '')
      .slice(0, 120);

    return this.prisma.product.create({
      data: {
        shopId: shop.id,
        title: input.title.trim(),
        slug: `${slugBase}-${randomUUID().slice(0, 8)}`,
        description: input.description?.trim() || null,
        category: input.category?.trim() || 'Autre',
        imageUrl: input.imageUrl ?? null,
        additionalImageUrls: input.additionalImageUrls ?? [],
        basePrice: input.basePrice,
        discountPrice: input.discountPrice ?? null,
        condition: input.condition ?? ProductCondition.NEUF,
        stockQuantity: input.stockQuantity,
        isPublished: shop.status === ShopStatus.ACTIVE,
      },
      include: { shop: { select: { id: true, name: true, status: true } } },
    });
  }
```

FIGURE 3.1 – Code Source : Service Métier de Publication de Produits & Contrôle des Stocks

prisma-orders.repository.ts — Calcul des Soldes & Retraits Vendeur

```ts
    const pendingByShop = await this.prisma.withdrawalRequest.groupBy({
      by: ['shopId'],
      where: {
        status: { in: ['PENDING', 'APPROVED', 'PROCESSING'] },
      },
      _sum: { amount: true },
    });

    const summary = shops.map((shop) => {
      const earned = shop.subOrders.reduce((sum, subOrder) => sum + Number(subOrder.commissionAmount), 0);
      const pendingWithdrawal = pendingByShop.find((entry) => entry.shopId === shop.id)?._sum.amount
        ? Number(pendingByShop.find((entry) => entry.shopId === shop.id)?._sum.amount ?? 0)
        : 0;
      const available = Math.max(earned - pendingWithdrawal, 0);
      return {
        shopId: shop.id,
        shopName: shop.name,
        earned,
        pendingWithdrawal,
        available,
      };
    });

    return {
      totalEarned: summary.reduce((sum, item) => sum + item.earned, 0),
      totalPendingWithdrawal: summary.reduce((sum, item) => sum + item.pendingWithdrawal, 0),
```

FIGURE 3.2 – Code Source : Agrégation Transactionnelle du Solde Marchand & Gestion des Retraits

Chapitre 4

Référentiel Exhaustif des 40 Use Cases du Super Admin

4.1 Domaine 1 : Pilotage Stratégique & Tableau de Bord

ID	Cas d'Usage	Spécification Fonctionnelle & Règles Métier
UC-1.1	Visualisation CA Global	Agrégation en temps réel du volume d'affaires total encaissé en FCFA sur l'ensemble des canaux.
UC-1.2	Suivi du Pipeline KYC	Ratio boutiques actives vs dossiers en attente d'audit de conformité.
UC-1.3	Compteur des Commandes	Taux de conversion, volume de paniers payés, et alertes de commandes bloquées.
UC-1.4	Grand Livre Commissions	Suivi du prélèvement net de 10% revenant à la marketplace Malick Buss.
UC-1.5	Flux Transactionnel Direct	Journal d'activité rafraîchi par polling ou SSE affichant les transactions minute par minute.

Table 4.1 – Référentiel Use Cases — Domaine 1 : Pilotage Stratégique

4.2 Domaine 2 : Onboarding & Validation KYC des Boutiques

ID	Cas d'Usage	Spécification Fonctionnelle & Règles Métier
UC-2.1	Revue de Conformité	Inspection de la pièce d'identité du gérant, registre NINEA et localisation physique.
UC-2.2	Validation Active	Passage du statut à `ACTIVE` : publication instantanée de la boutique sur la vitrine.
UC-2.3	Accueil WhatsApp Direct	Génération d'un message wa.me pré-formaté notifiant le gérant de l'ouverture de sa boutique.
UC-2.4	Verrouillage Caméra	Activation de `requireLiveCapture` obligeant le marchand à photographier ses produits en direct.
UC-2.5	Suspension Temporaire	Gel immédiat des ventes d'une boutique avec masquage automatique de tous ses produits.
UC-2.6	Rejet & Archivage	Notification motivée de refus de dossier pour dossier incomplet ou non-conformité.

Table 4.2 – Référentiel Use Cases — Domaine 2 : Onboarding & Vérification Marchande

4.3 Domaines 3 à 8 : Gestion Opérationnelle, Catalogue & Finance

Domaine	Cas Couverts	Objectif Métier & Impact
3. Vendeurs	UC-3.1 à UC-3.5	Répertoire téléphonique, contact d'assistance, audit de performance des boutiques.
4. Catalogue	UC-4.1 à UC-4.5	Modération a posteriori, dépublication d'articles contrefaits, contrôle de prix.
5. Commandes	UC-5.1 à UC-5.6	Surveillance des flux Wave/OM, arbitrage des contestations clients sous 7 jours.
6. Finances	UC-6.1 à UC-6.6	Validation des demandes de retraits ($\geq 5\,000$ FCFA), rapprochement des comptes de monnaie électronique.
7. Sécurité	UC-7.1 à UC-7.4	Gestion des comptes admin, révocation des tokens JWT, audit logs immuables.
8. Paramétrage	UC-8.1 à UC-8.4	Ajustement du taux de commission (10%), frais de livraison (3 000 FCFA), délais de réservation.

Table 4.3 – Référentiel Synthétique — Domaines 3 à 8

Chapitre 5

Qualité Logicielle, Pipeline CI/CD & SonarQube

5.1 Architecture Globale CI/CD & Bouclier Qualité

Le pipeline GitHub Actions déploie automatiquement les composants dès qu'un commit est fusionné sur `main`, sous réserve de valider l'ensemble des barrières de sécurité :

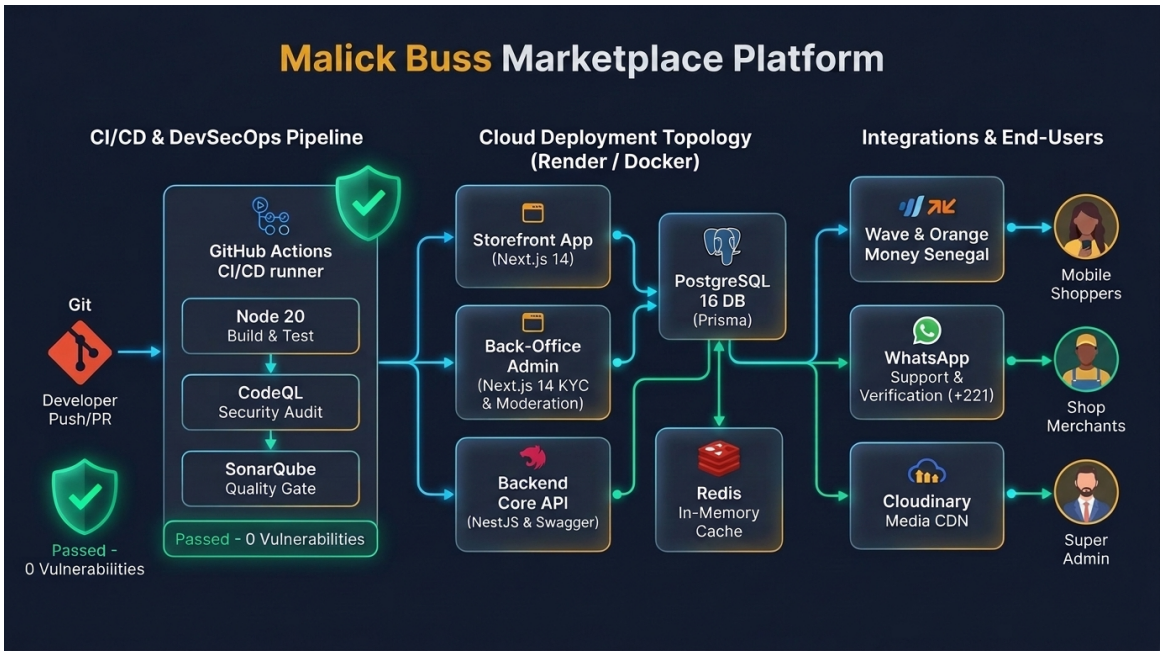


FIGURE 5.1 – Architecture Globale Cloud, Déploiement CI/CD & Bouclier Quality Gate SonarQube

5.2 Exigences du Quality Gate SonarQube

Le projet est gouverné par le fichier `sonar-project.properties` :

— **0 Vulnérabilités (Vulnerabilities) :** Aucune faille de sécurité tolérée en production.

— **0 Points Chauds (Security Hotspots) :** Tout flux sensible fait l'objet d'un examen systématique.

— **Couverture de Code** $> 80\%$ **:** Validée par les suites de tests unitaires et d'intégration API.

— **Dette Technique** $< 5\%$ **:** Respect strict des conventions TypeScript et proscription des types `any`.

Chapitre 6

Topologie de Déploiement Hybride & Écosystème Sénégal

6.1 Schéma Structurel du Flux Opérationnel

Le schéma suivant illustre le cheminement complet d'une transaction, de la commande client jusqu'au versement des fonds au vendeur :

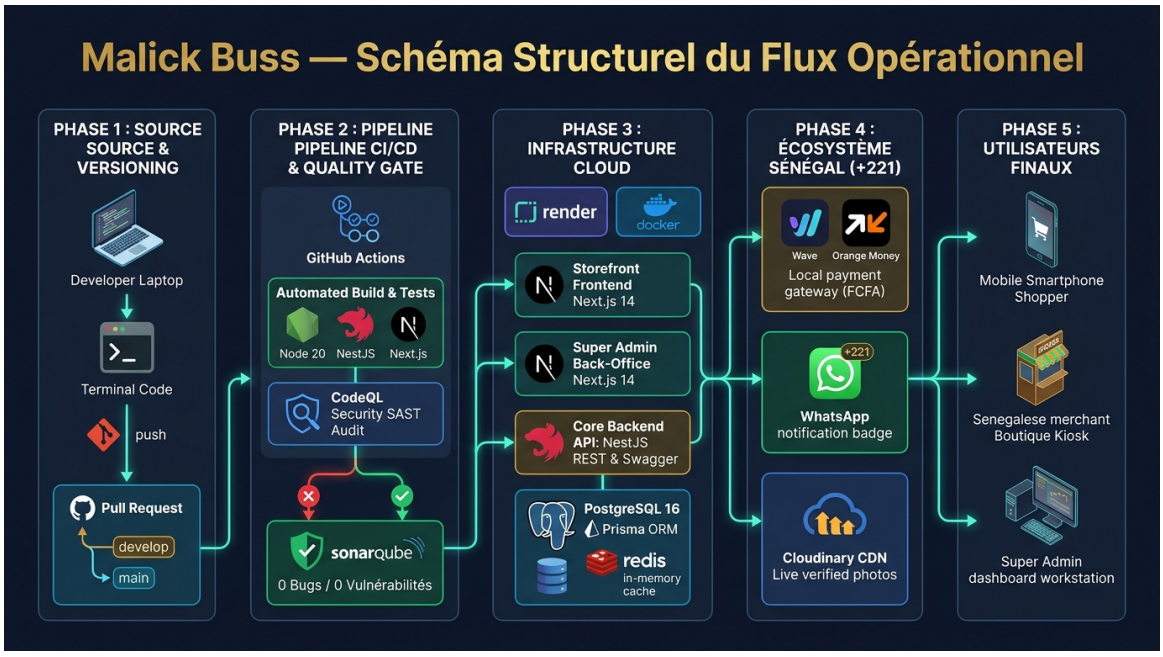


FIGURE 6.1 – Schéma Structurel du Flux Opérationnel en 5 Phases d'Exécution

6.2 Hébergement Hybride Déployé en Production

L'infrastructure actuelle tire parti des forces respectives des plateformes leaders :

— **Vitrine Cliente & Espace Vendeur (Vercel) :** Hébergée sur l'Edge Network mondial avec temps de chargement inférieur à 500 ms sur mobile à Dakar (https://frontend-malickdevwebs-projects.vercel.app).

— **Console Super Admin (Vercel) :** Interface de gouvernance privée sécurisée (https://back-office-malickdevwebs-projects.vercel.app).

— **API Backend NestJS (Render Frankfurt) :** Conteneur d'exécution Node.js résilient connecté aux passerelles de paiement (`https://malick-buss-api.onrender.com`).

— **Base de Données Neon Serverless PostgreSQL :** Stockage relationnel haute disponibilité avec basculement automatique et chiffrement des données au repos.

Chapitre 7

Galerie Visuelle des Interfaces & Maquettes

7.1 Parcours Acheteur & Espace Marchand Mobile-First

Les interfaces ont été conçues selon les standards ergonomiques les plus exigeants, optimisées pour la fluidité tactile sur smartphone :

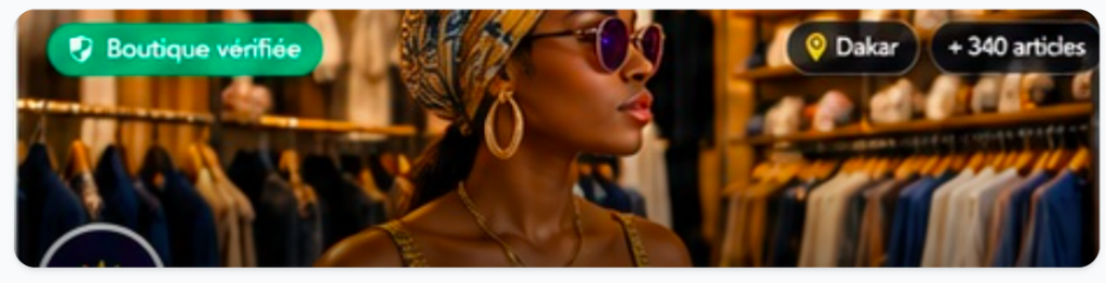


FIGURE 7.1 – Maquette Interface Utilisateur : Parcours d'Achat & Découverte Produits

7.2 Tableau de Bord Vendeur & Modération Super Admin

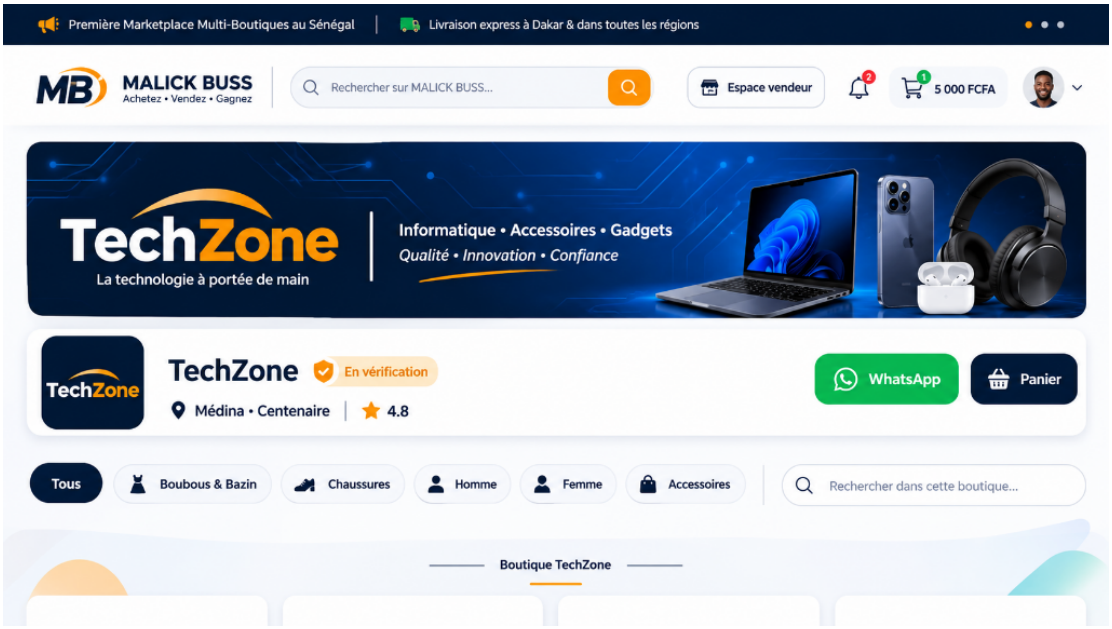


FIGURE 7.2 – Maquette Interface Marchand : Gestion des Ventes, Stocks & Demandes de Retrait

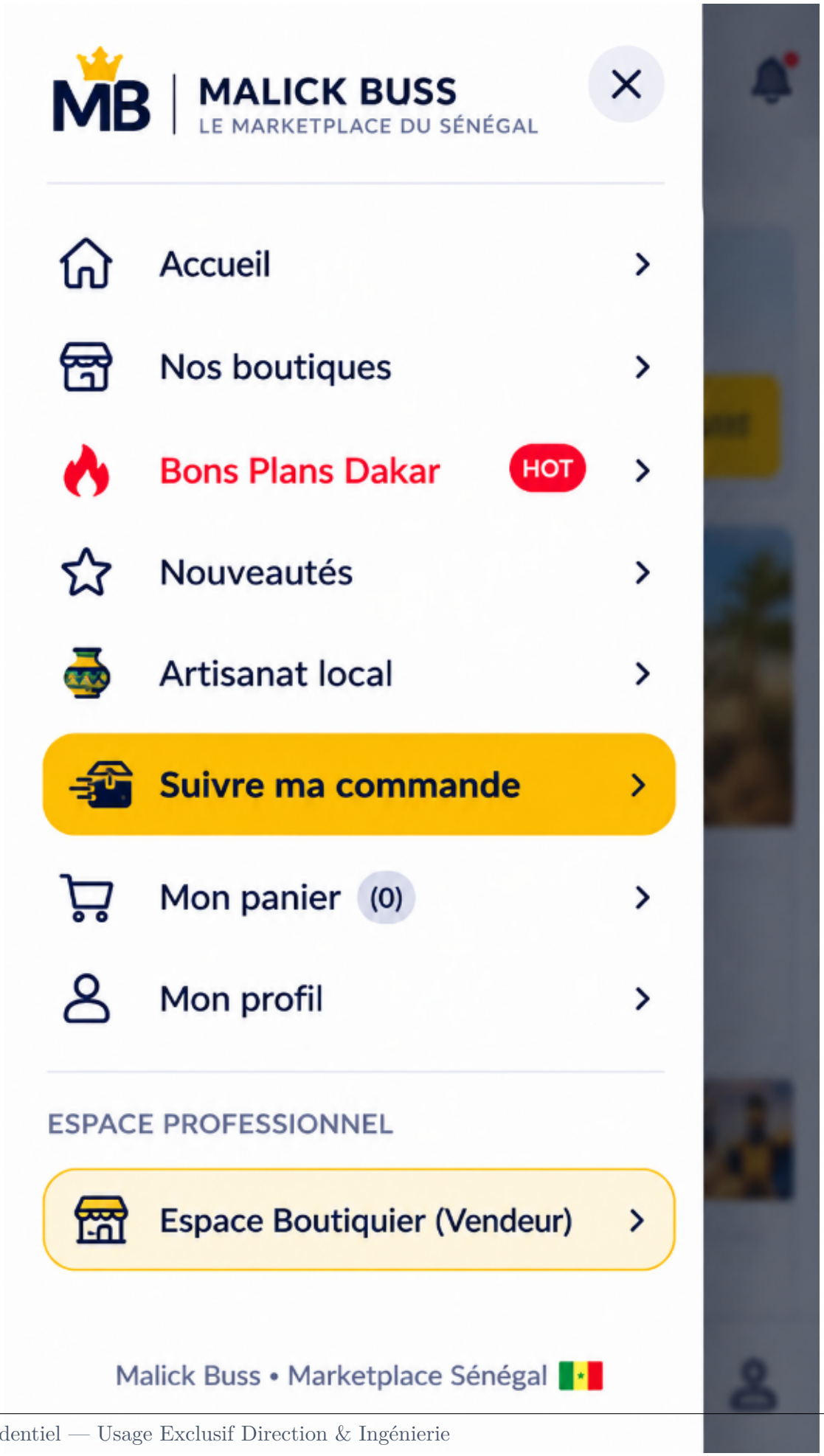


FIGURE 7.3 – Console Super Admin Mobile : Validation KYC & Arbitrage des Retraits de Fonds

Chapitre 8

Manuel d'Exploitation, Commandes & Plan de Continuité

8.1 Répertoire des Commandes Opérationnelles

Commande npm / CLI	Description & Effet Opérationnel
`npm run build:shared`	Compilation du paquet partagé TypeScript (`@malick-buss/shared`).
`npm run build:backend`	Compilation du binaire de production NestJS.
`npm run build:frontend`	Compilation Next.js de la vitrine client (40 pages statiques).
`npm run build:admin`	Compilation de la console d'administration Super Admin.
`npm run test:api -workspace=backend`	Validation des 4 suites de tests d'intégration API.
`npm run doc:build`	Compilation automatisée du présent dossier d'architecture en PDF.
`render services list`	Supervision des services cloud déployés sur Render.
`vercel deploy -prod`	Déploiement en production immédiat sur Vercel.

Table 8.1 – Manuel des Commandes d'Exploitation Quotidienne

8.2 Plan de Continuité d'Activité (PCA) & Reprise d'Urgence

En cas de sinistre ou d'indisponibilité d'un fournisseur cloud :

— **Sauvegarde Automatique de la Base de Données :** Neon effectue des sauvegardes continues avec restauration à la seconde près (Point-In-Time-Recovery / PITR).

— **Redémarrage Sans État :** Les applications Vercel et Render sont entièrement *stateless*. Une nouvelle instance peut être instanciée en moins de 3 minutes depuis le dépôt GitHub officiel.

— **Protection des Clés Secrètes :** Tous les secrets (JWT, Cloudinary, Wave) sont isolés dans les tableaux de bord respectifs et ne sont jamais committés dans le dépôt Git.

Document d'ingénierie et d'exploitation officiel — Malick Buss Sénégal © 2026.
Tous droits réservés. Reproduction et diffusion soumises à autorisation préalable de la Direction Technique.